JAVA 2

Module GEN

Types génériques

Au programme

1 La classe générique

2 Plusieurs paramètres

3 L'effacement

4 Les jokers

5 Méthodes génériques

6 L'héritage

1. Un type en paramètre



Quelqu'un a écrit ce `E`

Depuis COL, vous lisez les chevrons sans y penser

- Dans le JDK, l'auteur de `List` a écrit une lettre `E`

Dans ce module, vous allez apprendre à écrire ces classes “génériques”

```java
// ce que vous avez écrit dans le module COL
List<String> titres = new ArrayList<>();

// ce qu'a ecrit l'auteur de List (extrait simplifie)
public interface List<E> {
    boolean add(E element);
    E get(int index);
}
```

Démonstration



Démonstration

L'erreur qui change de camp

Déroulé : Demo_GEN

Déclarer un paramètre, le fixer à l'usage

<T> après le nom de la classe déclare un paramètre de type

T s'écrit ensuite partout où un type peut s'écrire

À l'usage, un argument de type prend sa place

```
public class Casier<T> {
    private T contenu;

    public void deposer(T objet) {
        contenu = objet;
    }

    public T retirer() {
        T objet = contenu;
        contenu = null;
        return objet;
    }
}

Casier<Valise> casier = new Casier<>();
```

Le casier contient des T, on peut y déposer un T et en retirer un T…

Un T se transmet

T peut servir d'argument à un autre type générique

La classe Consigne<T> enveloppe une collection (List<T>), et lui délègue le travail

La collection hérite de la vérification

```
public class Consigne<T> {
    private final List<T> depots = new ArrayList<>();

    public void deposer(T objet) {
        depots.add(objet);
    }

    public int nombreDeDepots() {
        return depots.size();
    }
}
```

Exercices



GEN.1 Une paire, un seul type

fascicule GEN

2. Autant de types qu'il en faut



Plusieurs paramètres

Un type générique peut concerner plusieurs paramètres, qui se déclarent alors entre parenthèses dans les mêmes chevrons

Chacun s'utilise indépendamment des autres

L'usage donne un argument à chacun, dans l'ordre

```
public class Ticket<K, V> {
    private final K numero;
    private final V objet;

    public Ticket(K numero, V objet) {
        this.numero = numero;
        this.objet = objet;
    }
}

Ticket<Integer, Valise> t = new Ticket<>(42, valise);
```

Un argument de type peut être générique

Un argument de type peut contenir lui-même des chevrons

On le lit de l'extérieur vers l'intérieur

Le compilateur vérifie chaque niveau

```
Map<String, List<Valise>> parVoyageur = new HashMap<>();

Casier<List<Valise>> casierFamille = new Casier<>();

Ticket<Integer, Casier<Valise>> ticket =
        new Ticket<>(7, new Casier<>());
```

Exercices



GEN.2 Un couple, deux types

fascicule GEN

GEN.3 Deux sortes de triplets

fascicule GEN

3. Ce que le compilateur efface



Une seule classe à l'exécution

Pour le compilateur, `Casier<Valise>` et `Casier<String>` diffèrent

À l'exécution, il n'existe qu'une classe `Casier`

Les arguments de type ont disparu : c'est l'effacement

```
Casier<Valise> pourValises = new Casier<>();
Casier<String> pourMessages = new Casier<>();

System.out.println(
        pourValises.getClass() == pourMessages.getClass());
// true
```

Vérifier, puis effacer

Le compilateur vérifie chaque usage de `T`

Puis il remplace `T` par `Object` dans la classe

Et il écrit lui-même le transtypage chez l'appelant

Vous écrivez…

```
Casier<Valise> casier = new Casier<>();
casier.deposer(new Valise("bleue"));
Valise v = casier.retirer();
```

Le compilateur transforme…

```
Casier casier = new Casier();
casier.deposer(new Valise("bleue"));
Valise v = (Valise) casier.retirer();
```

Ce que l'effacement interdit

Refusé	Parce qu'à l'exécution
`Casier<int>`	`T` devient `Object`, et un `int` n'est pas un objet
`new T()`	on ne sait plus quelle classe construire
`new T[10]`	un tableau doit connaître le type de ses cases
`static T champ;`	tous les casiers partagent une seule classe

Le transtypage qu'on ne peut pas éviter

Parfois, seul le programmeur sait ce que contient un `Object`

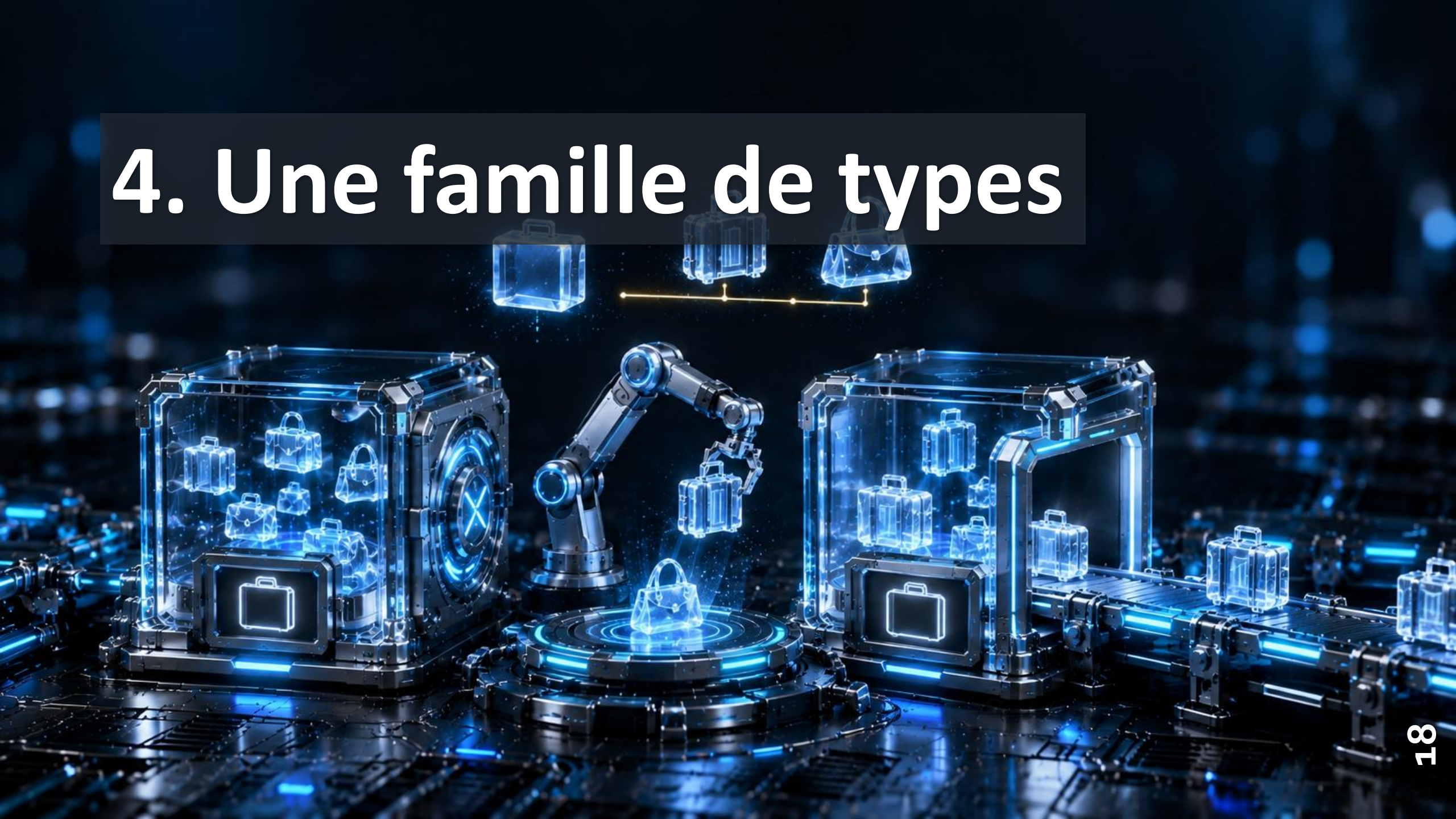
Le compilateur accepte le transtypage, mais ne peut pas le vérifier

`@SuppressWarnings("unchecked")` signe : je l'ai vérifié

```
// dans une classe generique, sur un tableau d'Object
@SuppressWarnings("unchecked")
T objet = (T) cases[i];

// au module FIC, en relisant un fichier
List<Valise> valises = (List<Valise>) entree.readObject();
```

4. Une famille de types



Une `Valise` est un `Bagage`

`Valise` et `Sac` héritent de `Bagage`

Une `List<Bagage>` accepte donc des valises et des sacs

Peut-on passer une `List<Valise>` à une méthode qui attend une `List<Bagage>` ?

```
// Valise extends Bagage, Sac extends Bagage

List<Bagage> soute = new ArrayList<>();
soute.add(new Valise("bleue"));       // accepte
soute.add(new Sac());                 // accepte
```

Exercices



GEN.4 Le joker

fascicule GEN

D'abord la version évidente

`? extends` : on lit

`? extends Bagage` : une liste de quelque chose qui est un `Bagage`

Une `List<Valise>` ou une `List<Sac>` y passent

On en lit des `Bagage`, on n'y ajoute rien

```
public static double poidsTotal(List<? extends Bagage> bagages) {
    double total = 0;
    for (Bagage b : bagages) {
        total += b.getPoids();
    }
    // bagages.add(new Sac());   ne compile pas
    return total;
}
```

`? super` : on ajoute

`? super Valise` : une liste de quelque chose dont `Valise` hérite

Une `List<Valise>`, `List<Bagage>` ou `List<Object>` y passent

On y ajoute des valises, on n'en lit que des `Object`

```
public static void charger(List<? super Valise> soute, int n) {
    for (int i = 0; i < n; i++) {
        soute.add(new Valise("grise"));
    }
}

charger(new ArrayList<Bagage>(), 3);    // accepte
```

`? extends` ou `? super`

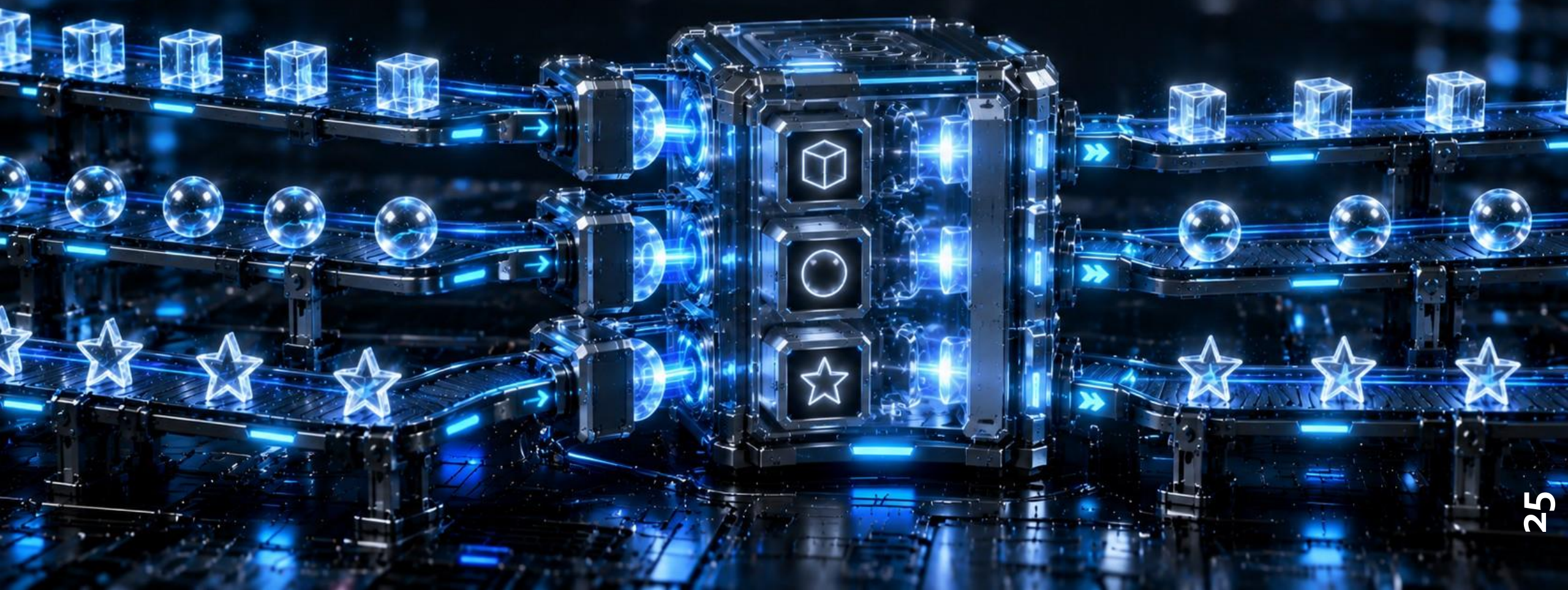
Tout dépend de ce que la méthode fait de la liste : la lire, ou la remplir.

	`? extends Bagage`	`? super Valise`
Accepte	`Bagage`, `Valise`, `Sac`	`Valise`, `Bagage`, `Object`
Lire	oui, des `Bagage`	des `Object` seulement
Ajouter	non	oui, des `Valise`
La liste	fournit	reçoit
Exemple	`poidsTotal(...)`	`charger(...)`

Ce que chaque joker accepte

`List<? extends Bagage>`		`List<? super Valise>`	
on y lit des Bagage		*on y ajoute des Valise*	
`List<Bagage>`	*acceptée*	`List<Valise>`	*acceptée*
`List<Valise>`	*acceptée*	`List<Bagage>`	*acceptée*
`List<Sac>`	*acceptée*	`List<Object>`	*acceptée*
`List<Object>`	*refusée*	`List<Sac>`	*refusée*

5. Une méthode générique



Le compilateur déduit le T

Le T d'une méthode se fixe à chaque appel

Le compilateur le déduit des arguments

On peut le donner soi-même, mais c'est rarement utile

```
public static <T> T auHasard(List<T> elements) {
    int i = (int) (Math.random() * elements.size());
    return elements.get(i);
}

Valise v = Tirage.auHasard(valises);    // T vaut Valise
String s = Tirage.auHasard(messages);   // T vaut String
Valise w = Tirage.<Valise>auHasard(valises);
```

Une méthode statique déclare son T

Le T d'une classe appartient à chacun de ses objets

Une méthode `static` n'a pas d'objet : elle ne le voit pas

Elle déclare le sien, comme cette fabrique

```
public class Casier<T> {
    // ...

    public static <T> Casier<T> contenant(T objet) {
        Casier<T> casier = new Casier<>();
        casier.deposer(objet);
        return casier;
    }
}

Casier<Valise> casier = Casier.contenant(valise);
```

Exercices



GEN.5 Une méthode générique

fascicule GEN

6. Hériter d'une classe générique



Deux façons d'hériter

Une fille peut rester générique : `extends Casier<T>`

Ou fixer le type une fois pour toutes : `extends Casier<Velo>`

Les méthodes héritées suivent l'argument donné à la mère

```
public class CasierFerme<T> extends Casier<T> {
    private final String code;

    public CasierFerme(String code) {
        this.code = code;
    }
}

public class CasierVelo extends Casier<Velo> {
    // deposer(Velo) et retirer() : Velo, herites tels quels
}
```

Une interface générique

Une interface aussi peut être générique : `Depot<T>`

Elle déclare ici `deposer(T)` et `retirer()`, qui retourne un `T`

La classe qui l'implémente transmet le type, ou le fixe

```
public class Vestiaire implements Depot<Manteau> {
    private Manteau accroche;

    @Override
    public void deposer(Manteau m) { accroche = m; }

    @Override
    public Manteau retirer() { return accroche; }
}
```

Exercices



GEN.6 Hériter d'une classe générique

fascicule GEN

Code commun : CC-CLS.3, tests : CC-MET.7

Le fil rouge



G.31 Un objet par joueur

fil rouge GameHub

Deux champs jumeaux deviennent un objet